

Do Published HumanEval Rankings Survive Local Deployment?

A Five-Model Reproduction on a Consumer GPU

Jaiveer Bassi

Independent Researcher

jaiveerbassi@yahoo.com

Technical report – primary experiment

Version 1.0, September 14, 2026

Abstract

Published benchmark tables are often reused as if model rankings were properties of the checkpoints alone. In practice, a score also depends on prompt construction, decoding, stopping, software versions, and execution policy. We tested whether the HumanEval ordering of five open base checkpoints reported together in Table 5 of the Qwen2.5-Coder technical report survives a uniform local deployment condition. Each checkpoint produced one greedy completion for all 164 HumanEval tasks in BF16, at batch size one, on a single NVIDIA RTX 4070. EvalPlus 0.3.1 scored the same completions against HumanEval and HumanEval+ inside a network-isolated container. Four models retained their relative order, but StarCoder2-3B’s published HumanEval score of 31.7% corresponded to 1.8% locally, reversing its ordering with Qwen2.5-Coder-0.5B. Kendall’s tau-b between published and local HumanEval rankings was 0.8 (task-bootstrap 95% interval: 0.6 to 0.8). The paired local pass@1 difference for StarCoder2-3B minus Qwen2.5-Coder-0.5B was -21.95 percentage points (task-bootstrap 95% interval: -28.66 to -15.85), satisfying the preregistered failure-to-reproduce rule. A diagnostic indicates prompt-and-stopping sensitivity: the pinned StarCoder2 checkpoint can generate ordinary code for its model-card prompt, but under the uniform HumanEval condition it can repeat a top-level function definition that the evaluator treats as a stop boundary. These results do not show that the published score is erroneous. They show that one published ranking did not transfer to a fully specified, auditable local condition and that deployment details can materially alter a nominal model comparison.

Keywords: code language models; HumanEval; EvalPlus; reproducibility; local inference; functional correctness

1. Introduction

HumanEval measures whether generated Python functions satisfy hidden tests and remains a common point of comparison for code language models [Chen et al., 2021]. HumanEval+ strengthens that evaluation with ad-

ditional automatically generated tests [Liu et al., 2023]. Model reports commonly place scores from several checkpoints in one table, inviting readers to interpret the displayed order as a portable ranking.

That interpretation can be fragile. Functional-correctness results depend on more than weights: prompt templates, stopping strings, generation length, precision, library behavior, and sanitization can all change the executable completion. These choices matter especially for base models, whose training formats and preferred continuation patterns differ.

We reproduce an external and inspectable target: the ordering of five base checkpoints in Table 5 of the Qwen2.5-Coder technical report [Hui et al., 2024]. The target contains Qwen2.5-Coder at 0.5B, 1.5B, and 3B parameters, DeepSeek-Coder-1.3B, and StarCoder2-3B. The model families are documented separately by their developers [Hui et al., 2024, Lozhkov et al., 2024, Guo et al., 2024]. Our question is deliberately narrower than exact score replication: does the published HumanEval ordering survive when all five checkpoints are run through one pinned local stack on a consumer GPU?

The contribution is a completed five-model primary experiment with a predeclared rank endpoint, task-level paired uncertainty, exact checkpoint revisions, raw and sanitized completions, full evaluator outputs, and environment metadata. The principal finding is one statistically resolved rank reversal. The repository contains the evidence needed to audit that finding at task level [Bassi, 2026].

2. Related Work

HumanEval introduced execution-based functional correctness for 164 handwritten Python synthesis tasks and the pass@ k family of estimators [Chen et al., 2021]. EvalPlus subsequently expanded the tests associated with HumanEval and showed that test insufficiency can change both absolute scores and model orderings [Liu et al., 2023]. Our study uses the released HumanEval+ tests but asks a different question: whether a published order transfers across deployment conditions when check-

point identity and functional tests are held fixed.

The Qwen2.5-Coder report compares several open base models under one table, including all five checkpoints studied here [Hui et al., 2024]. StarCoder2 and DeepSeek-Coder are independently documented model families with different training corpora, objectives, and evaluation practices [Lozhkov et al., 2024, Guo et al., 2024]. We do not construct a new leaderboard or claim contemporary model coverage. We treat the published table as a concrete external reproduction target and focus on the otherwise underreported inference boundary between weights and executable benchmark candidates.

3. Research Question and Decision Rule

The primary question is: **Does the HumanEval ordering of the five selected checkpoints in Qwen2.5-Coder Table 5 survive the specified local deployment condition?** HumanEval+ is co-reported as a robustness outcome, not substituted for the primary endpoint.

The primary estimand is Kendall’s tau-b between the published HumanEval order and the local greedy pass@1 order. We bootstrap tau-b and compare each adjacent pair in the published order using paired resampling over the 164 tasks. Each resample preserves all five model outcomes for a selected task. We use 10,000 bootstrap replicates and fixed analysis seed 2026.

The predeclared decision has three outcomes:

- **Reproduced:** the point ordering is identical, with tau-b equal to 1 and no adjacent published pair reversed locally.
- **Failed to reproduce:** at least one pair is reversed and the paired 95% bootstrap interval for that pass@1 difference excludes zero in the reversed direction.
- **Inconclusive:** every other outcome, including unresolved ties or reversals whose interval includes zero.

This endpoint tests rank transfer, not equality of percentages. Absolute score differences are descriptive because the source paper does not expose every decoding, stopping, and software detail needed to reconstruct Table 5 from the paper alone.

4. Methods

4.1 Models and immutable inputs

Table 1 lists the exact base checkpoints and revisions. Revisions were resolved and frozen before aggregate correctness results were inspected.

4.2 Generation condition

All five models used the EvalPlus base-model HumanEval prompt, with base mode forced even when a tokenizer exposed a chat template. Generation was greedy with

one completion per task, a maximum of 512 new tokens, batch size one, and BF16 weights. The models ran sequentially on one NVIDIA GeForce RTX 4070 with 12,282 MiB dedicated memory in WSL Ubuntu on Windows. The generation environment used PyTorch 2.8.0+cu128 and Transformers 4.57.6. No latency or throughput conclusion is drawn from the correctness run.

The primary design uses no repeated random seeds. Conditional on the frozen inputs and software stack, greedy decoding is deterministic; repeated identical runs would measure system nondeterminism rather than sampling variance. A separately specified 20-sample sensitivity experiment has not been run and is not reported here.

4.3 Functional evaluation and execution safety

The same completions were evaluated on all 164 HumanEval tasks and the corresponding HumanEval+ v0.1.10 tests. We pinned EvalPlus 0.3.1 at commit e5d0ed0bab96280b60b637ec7f15b5e4841b0cb2. Generated Python was executed only in a disposable Linux container with networking disabled, credentials absent, capabilities dropped, and no writable host mount beyond the result exchange directory.

The official Docker image tagged v0.3.1 exposed a development package version during verification. We therefore derived the evaluator from the official immutable base-image digest, force-installed released evalplus==0.3.1, asserted the installed version at build time, and recorded the derived image identifier.

4.4 Harness equivalence checks

Two changes addressed avoidable generation overhead without changing the declared stopping predicate. First, the upstream batch-one stop criterion decoded the same completion once for each stop string after every token. The wrapper decodes once and checks all stop strings; a pinned one-task comparison required byte-identical raw and sanitized outputs before the full run. Second, EvalPlus left a wrapped Transformers stopping method installed after each task, causing criteria to accumulate. A `finally` block restores the original method after generation. We subsequently repeated HumanEval/0 and HumanEval/1 through unmodified EvalPlus for all five checkpoints. Raw and sanitized records were byte-identical to the retained primary records for every model and task. Machine-readable hashes for both checks are included in the repository.

The provider’s mutable stop list could also acquire duplicate strings when models were constructed sequentially. The runner removes only exact duplicates while preserving first occurrence and order, leaving the logical `any(stop in decoded)` predicate unchanged.

Table 1: Frozen model inputs. Full identifiers also appear in machine-readable protocol metadata.

Model	Hugging Face checkpoint	Revision
Qwen2.5-Coder-0.5B	Qwen/Qwen2.5-Coder-0.5B	8123ea2e9354afb7ffcc6c8641d1b2f5ecf18301
StarCoder2-3B	bigcode/starcoder2-3b	733247c55e3f73af49ce8e9c7949bf14af205928
DeepSeek-Coder-1.3B	deepseek-ai/deepseek-coder-1.3b-base	c919139c3a9b4070729c8b2cca4847ab29ca8d94
Qwen2.5-Coder-1.5B	Qwen/Qwen2.5-Coder-1.5B	df3ce67c0e24480f20468b6ef2894622d69eb73b
Qwen2.5-Coder-3B	Qwen/Qwen2.5-Coder-3B	09d9bc5d376b0cfa0100a0694ea7de7232525803

4.5 Analysis

Pass@1 is the proportion of 164 tasks for which the single completion passed. At one completion per task this is a direct proportion, not the multi-sample estimator introduced with HumanEval [Chen et al., 2021]. Bootstrap intervals are computed on paired task outcomes. The analysis program rejects missing, duplicate, non-Boolean, or mismatched task records before aggregation.

5. Results

Table 2 reports all primary measurements. Exact counts are included to avoid false precision.

The published HumanEval order from low to high was Qwen2.5-Coder-0.5B, StarCoder2-3B, DeepSeek-Coder-1.3B, Qwen2.5-Coder-1.5B, and Qwen2.5-Coder-3B. Locally, StarCoder2-3B moved below Qwen2.5-Coder-0.5B; the other four positions retained their relative order. Kendall’s tau-b was 0.8, with a paired task-bootstrap 95% interval of [0.6, 0.8].

For the reversed adjacent pair, the local difference for StarCoder2-3B minus Qwen2.5-Coder-0.5B was -21.95 percentage points, with paired-bootstrap 95% interval [-28.66, -15.85]. The interval excludes zero in the reversed direction. The primary decision is therefore **failed to reproduce**.

The other adjacent local HumanEval differences, in published low-to-high order, were: DeepSeek-Coder-1.3B minus StarCoder2-3B, +32.32 points [25.61, 39.63]; Qwen2.5-Coder-1.5B minus DeepSeek-Coder-1.3B, +4.88 points [-2.44, 12.20]; and Qwen2.5-Coder-3B minus Qwen2.5-Coder-1.5B, +13.41 points [5.49, 21.34]. These intervals are descriptive and do not alter the predeclared decision once a resolved reversal is present.

HumanEval+ preserved the same local order. Qwen2.5-Coder-3B matched the displayed published percentage to rounding on both benchmarks; DeepSeek-Coder-1.3B was 1.9 points higher locally on HumanEval+; and the remaining Qwen checkpoints were 2.5 to 4.9 points lower. StarCoder2-3B was the clear outlier, 29.9 points below the displayed HumanEval score and 25.6 points below the displayed HumanEval+ score.

6. Diagnostic Analysis

The StarCoder2-3B result warranted a targeted diagnostic because its absolute difference was much larger than those of the other models. Using the same pinned weights, BF16 precision, and eager Transformers path, the checkpoint generated ordinary Python for the short prompt shown in its model card [BigCode, 2024]. For the HumanEval/0 prompt, however, it began by repeating a complete top-level function definition. EvalPlus treats a new top-level `def` as a completion boundary. The sanitized candidate therefore contained no function body and failed. The retained task artifacts show a broader pattern of prompt-only or non-solution continuations, consistent with format sensitivity.

This diagnostic narrows the interpretation but does not establish a single cause. It shows that the checkpoint can generate syntactically plausible code and that the observed failure is compatible with an interaction among prompt format, learned continuation style, and stop policy. We did not alter the prompt or stopping rules after inspecting scores because that would change the primary condition. A controlled prompt-and-stop ablation belongs in the planned sensitivity study.

7. Threats to Validity

Target specification. The published percentages and model identities are available in Qwen2.5-Coder Table 5, but the report does not specify every generation and post-processing detail for that table. The current public evaluation-code link associated with the report is not, by itself, an immutable reconstruction of the historical run. Our experiment therefore tests transfer to a disclosed condition, not faithful rerun of an undisclosed condition.

Single deployment stack. We used one GPU, one precision, one inference library version, and one evaluator release. The result does not estimate variation across hardware, kernels, libraries, or quantization modes. It also does not support CPU performance claims.

One greedy completion. The primary endpoint measures deterministic greedy pass@1. It does not characterize sampling distributions or pass@5. The task bootstrap quantifies uncertainty across the finite task set; it is not a substitute for repeated stochastic completions.

Benchmark scope. HumanEval contains 164 Python tasks. HumanEval+ adds tests but not independent

Table 2: Published percentages from Qwen2.5-Coder Table 5 and local primary results. HE denotes HumanEval; HE+ denotes HumanEval+.

Model	Published HE	Local HE	Published HE+	Local HE+
Qwen2.5-Coder-0.5B	28.0%	23.8% (39/164)	23.8%	20.1% (33/164)
StarCoder2-3B	31.7%	1.8% (3/164)	27.4%	1.8% (3/164)
DeepSeek-Coder-1.3B	34.8%	34.1% (56/164)	26.8%	28.7% (47/164)
Qwen2.5-Coder-1.5B	43.9%	39.0% (64/164)	36.6%	34.1% (56/164)
Qwen2.5-Coder-3B	52.4%	52.4% (86/164)	42.7%	42.7% (70/164)

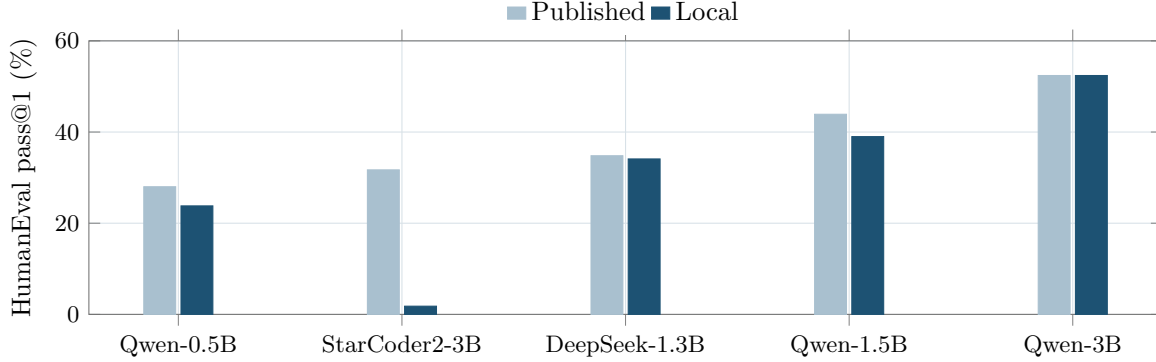


Figure 1: Published and local HumanEval pass@1. The StarCoder2-3B divergence determines the rank-reproduction outcome.

tasks, so it does not increase the task-level sample size. We did not audit benchmark contamination, language generality, security, maintainability, or real repository-level coding ability.

Diagnostic scope. The StarCoder2 probe was selected after observing the primary outcome and is explanatory rather than confirmatory. The data support sensitivity under this harness, not a claim that StarCoder2 is generally inferior or that its published score is false.

Temporal scope. We reproduce a 2024 comparison using frozen revisions of the named historical checkpoints because the research question concerns transfer of that published ordering. This is not a claim that the five models represent the strongest code models available in 2026.

8. Reproducibility and Artifact Availability

The public repository [Bassi, 2026] contains:

- the preregistered question, endpoint, decision rule, and conditional secondary plan;
- five immutable checkpoint revisions and the published target table in JSON;
- raw and sanitized completions for all 820 model-task pairs;
- five complete evaluator result files and 820 Boolean task outcomes;
- the generated analysis JSON, runtime package freeze,

GPU metadata, Git state, evaluator image identifier, and SHA-256 checksums;

- the generation, validation, consolidation, analysis, and hardened evaluation code; and
- equivalence records for the generation-wrapper changes, including two-task stock controls for all five models.

The artifact directory contains model-generated Python and should be treated as untrusted data. Re-evaluation should use the documented disposable container rather than executing samples on a host system. The repository is public, but no DOI or third-party archival identifier is claimed in this version.

9. Conclusion

The published five-model HumanEval ordering did not survive our uniform local deployment condition. A single resolved reversal reduced Kendall’s tau-b to 0.8 (task-bootstrap 95% interval: 0.6 to 0.8) and met the preregistered failure-to-reproduce rule. The result was driven by StarCoder2-3B, whose generated continuation interacted poorly with the common HumanEval prompt and stopping policy. Four other checkpoints were substantially closer to the displayed source scores, including a near-exact rounded match for Qwen2.5-Coder-3B.

The practical lesson is not that published tables are unusable. It is that their order should not be treated as checkpoint metadata. For base code models, the benchmark result is a property of the complete evaluation system.

Reports should preserve prompt construction, decoding, stopping, sanitization, evaluator version, checkpoint revision, and task-level outputs together. The next confirmatory step is the predeclared sensitivity study, followed by controlled prompt and stop-policy ablations.

References

- Jaiveer Bassi. Do published HumanEval rankings survive local deployment? Software and research artifacts, version 1.0.0, 2026. URL <https://github.com/imjbassi/Local-Deployment-of-Transformer-based-Coder-Assistants>.
- BigCode. StarCoder2-3B model card. Hugging Face, 2024. URL <https://huggingface.co/bigcode/sstarcoder2-3b>. Accessed 2026-09-14.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. URL <https://arxiv.org/abs/2107.03374>.
- Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Yu Wu, Y. K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. DeepSeek-Coder: When the large language model meets programming – the rise of code intelligence. *arXiv preprint arXiv:2401.14196*, 2024. URL <https://arxiv.org/abs/2401.14196>.
- Binyuan Hui, Jian Yang, Zeyu Cui, Jiayi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. Qwen2.5-Coder technical report. *arXiv preprint arXiv:2409.12186*, 2024. URL <https://arxiv.org/abs/2409.12186>.
- Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems*, 2023. URL <https://arxiv.org/abs/2305.01210>.
- Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, et al. StarCoder 2 and The Stack v2: The next generation. *arXiv preprint arXiv:2402.19173*, 2024. URL <https://arxiv.org/abs/2402.19173>.